

Tuberculosis Detection of Chest X-Ray Images

Using Digital Image Processing Tools & Machine Learning

Muhammad Gibran Basyir*, Muhammad Razan Alamudi*,
Daan Andries Jacob Eeltink*, Devina Rahmadhita Dewantoro+, Aurora Cindita Syachbudtari+
“ * ” = 1st Highest Contribution, “ + ” = 2nd Highest Contribution

Faculty of Mathematics and Natural Science Gadjah Mada University, Yogyakarta, Indonesia

Abstract—Automated detection of Tuberculosis (TB) from chest radiographs is a critical area of research for improving diagnostic speed and accessibility. This project details the design and implementation of a machine learning pipeline to classify chest X-ray images as either ‘Normal’ or containing signs of ‘Tuberculosis’. The methodology begins with a robust image processing pipeline where raw images from a public X-ray dataset are first enhanced using Contrast Limited Adaptive Histogram Equalization (CLAHE) and median filtering. Subsequently, the lung fields are segmented using Otsu’s thresholding and morphological operations, incorporating a key step to ensure the separation of the left and right lungs. From each segmented lung, a vector of geometric, intensity, and texture-based-features, including area, mean intensity, and Shannon entropy is extracted. These features are then used to train a Random Forest Classifier. The model, evaluated on an unseen test set comprising 20% of the data, achieved a high overall accuracy of 92.76%. The results demonstrate that a combination of targeted image feature extraction and a machine learning approach can effectively serve as a reliable tool for automated Tuberculosis screening.

Keywords—Digital Image Processing, Tuberculosis Detection, Chest X-Ray Classification, Feature Extraction, Machine Learning, Random Forest, Image Segmentation.

I. INTRODUCTION

Tuberculosis remains one of the leading causes of mortality worldwide, specifically in low and middle income countries where access to advanced hospital tools are limited. Chest X-ray imaging is a widely available, low-cost method for TB screening, but interpretation/identification requires experienced radiologists. While some approaches use deep learning to automate TB detection, it may not be feasible in all environments. This project proposes an alternative, feature-based machine learning pipeline that is computationally efficient and provides interpretable results. The methodology focuses on a structured, multi-stage process. First, raw chest radiographs undergo a pre-processing stage involving Contrast Limited Adaptive Histogram Equalization (CLAHE) and median filtering to enhance image quality and reduce noise. Second, the lung fields are automatically segmented from the enhanced images. Then, a set of numerical features, based on geometry, intensity, and texture are extracted from the segmentations. These features form the basis for training a Random Forest classifier to identify between Normal and Tuberculous cases.

II. DATASET AND IMAGE PRE-PROCESSING

A. Dataset

The project utilizes the public "Tuberculosis (TB) Chest X-ray Dataset" available on Kaggle [1]. The posterior-anterior chest radiographs in this dataset have been carefully selected for the purpose of classifying tuberculosis. There are 4,200 images in all, divided into two different classes:

Normal: 3,500 images of healthy individuals.

Tuberculosis: 700 images showing radiological evidence of tuberculosis

The dataset is a good choice for training and testing a supervised machine learning model because it is well-labelled and substantial in size. We used the Kaggle API to download the dataset straight into Google Colab so that it could be processed efficiently.

B. Image Pre-processing

Before segmenting the lung regions, a pre-processing pipeline was applied to each raw image. The main goal of this step is to make the images more uniform and improve the visibility of lung structures, which is crucial for the accuracy of subsequent feature extraction. As shown in Fig. 1, the pipeline consists of two sequential steps:

Contrast Enhancement: We used Contrast Limited Adaptive Histogram Equalization (CLAHE) to fix differences in exposure and light levels between different X-ray images. Unlike global histogram equalization, CLAHE works on small regions of the image, called tiles. This enhances local contrast, making the edges of the lung fields and subtle textures within the lung parenchyma more distinct without over-amplifying the noise.

Noise Reduction: Chest radiographs often contain impulse noise (salt-and-pepper noise). To counteract this, the contrast-enhanced image was subjected to a 5x5 Median Filter. This non-linear filtering technique is highly effective at

removing such noise while maintaining the sharp edges of the anatomical structures, which is a significant advantage over linear filters like the Gaussian blur in this context.

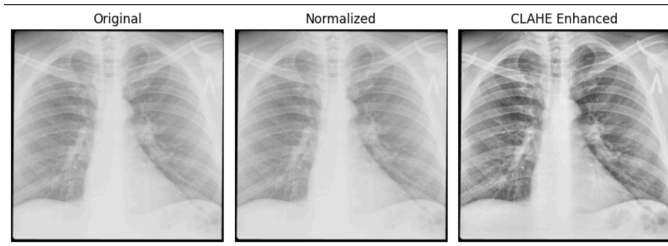


Figure 1: From Normal to CLAHE Enhanced



Figure 2: After applying both filtering and sharpening

III. LUNG SEGMENTATION AND FEATURE EXTRACTION METHODOLOGY

Following the pre-processing phase, a two part methodology was implemented to first isolate the lung regions and then to quantify them using a set of descriptive features. This process is the core of the image analysis pipeline, converting visual information into a structured, numerical format suitable for machine learning.

C. Lung Segmentation

The main objective of segmentation is to create a binary mask that accurately delineates the lung fields, separating them from the surrounding anatomy such as the heart, spine and diaphragm. The implemented pipeline is shown in Figure 3. Otsu's thresholding method was applied. This adaptive technique automatically calculates the optimal threshold value to separate the image into foreground and background, effectively creating a binary image. The resulting mask was then inverted, as the lung fields are typically darker (lower pixel intensity) in a standard radiograph. To refine the binary mask, a series of morphological operations are used. First, a morphological opening (erosion followed by dilation) was applied with a small 3x4 kernel to remove small, non-structural noise and artifacts, such as rib cages that were incorrectly included in the initial mask. Following this a morphological closing (dilation followed by erosion) with a larger 7x7 kernel was used to fill small holes and gaps within the main lung regions, creating a more solid and contiguous mask. After cleaning, a connected components analysis was performed to identify all separate white regions in the mask.

Then we continued by making separate masks for the left and right lung, and including the combined one.

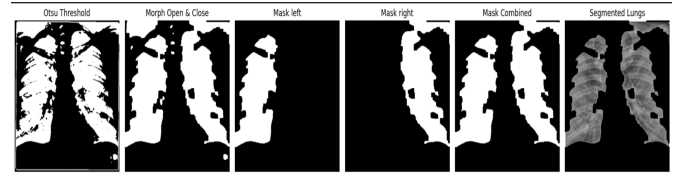


Figure 3: Lung Segmentation

D. Feature Extraction

Once a clean, separated mask of the two lungs were gathered, a feature extraction function was used to calculate a vector of numerical descriptors for each lung individually. These features are designed to capture information about the size, shape, and texture of the lung parenchyma.

1. *Geometric Features:* These features describe the basic morphology of each lung region.
 - Area: The total pixel count of the lung region, representing its size.
 - Perimeter: The length of the boundary of the lung region.
2. *Intensity and Texture Features:* These features quantify the distribution of intensity or pixel values within the lung regions, which is important for detecting pathological abnormalities.
 - Mean Intensity: The average pixel brightness within the lung. Changes in this value can indicate widespread inflammation or fluid.
 - Standard Deviation of Intensity: A measure of the contrast within the lung tissue. Higher values suggest a greater variation between light and dark patches, indicating a more complex texture
 - Shannon Entropy: A statistical measure of the randomness and complexity of the pixel intensity distribution. A greater entropy value indicates a more unpredictable and texturally complex region, which can be an indicator of disease.
3. *Between Lung Difference Features:* These meta-features describe how the values of the geometric, intensity and texture features differ between the left and the right lung. These are particularly important when detecting infections that only affect a single lung hemisphere, thereby causing a significant visual difference between the two lungs.

The extraction process results in a set of these five features for both the left and right lung, which are then passed to the machine learning stage. Fig. 4 demonstrates the final output of this stage, showing the bounding boxes identifying the two distinct lung regions from the feature extraction.

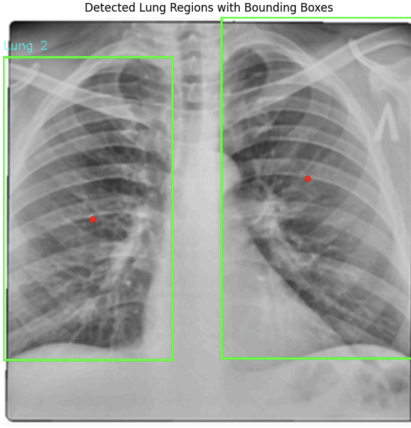


Figure 4: Visual output of the feature extraction stage, showing bounding boxes and centroids for the individually identified left and right lungs.

	label	area	perimeter	centroid	mean_intensity	std_intensity	entropy
0	1	51610.0	2230.212337	(202.8386359232707, 378.44316992830846)	122.519531	26.347882	6.369244
1	2	46967.0	1581.477272	(253.70832712329934, 111.89327400089425)	123.418656	22.782068	6.268901

Figure 5: Details of the feature extraction visual output

IV. MODEL RESEARCH AND SELECTING

Now that the digital image processing is complete with the feature successfully extracted with the visual output, we reached a stage where we search for the different machine learning models and choose the most optimal one. The selection of the machine learning model is crucial since not all models can be accurate for every method.

E. Rationale for Selecting Random Forest Classifier

We chose Random Forest Classification as our primary machine learning algorithm for this project, mainly due to its robustness, powerful capabilities, and overall suitability for this type of dataset. Some of the key advantages that made it an ideal choice include:

- **High Accuracy and Robustness to Overfitting:** Since Random Forest is an ensemble model, it's independent of a single, flawed decision-maker. It takes a majority vote from hundreds of individual decision trees to make a final prediction. This process prevents overfitting that is usually seen in single decision trees, and thus the model is able to generalize much better to new, unseen data - as evidenced by the high test accuracy.
- **The Lack of Need for Feature Scaling:** This classifier works well even when the dataset features have significantly different scales. For instance, one feature (left area) might be large, whilst another (entropy_diff) might be very small. Random Forest's tree-based structure is capable of handling raw values directly by comparing thresholds (e.g. is left_area > 40,000?). This makes our model a lot more robust and easier to implement.

- **Built-in Feature Importance:** By utilizing Random Forest, we are able to automatically identify which features most influence its decisions, which then helps us understand *why* the model works. For example, it might indicate that entropy_diff and right_mean_intensity are the strongest predictors. This gives us scientific insight by revealing exactly which of the extracted features are the most relevant in the context of detecting TB.
- **Ability to Handle Complex, Non-Linear Relationships:** In this case, the relationship between features (e.g., mean_intensity and entropy) and the outcome typically wouldn't be a simple straight line. For instance, TB might only appear when both features are high, something linear models would easily miss. Random Forest's tree-based structure captures these "if-this-and-that" patterns, making it very well-suited for complex medical data.

V. MAIN TECHNICAL CHALLENGES

F. File and Path Management (The 0 samples Error)

- **Problem:** When we first ran the "Main Loop," it processed 0 samples. The code couldn't find any of the images.
- **Mistake:** The file paths in NORMAL_PATH and TB_PATH were pointing to the folders (eg.../Normal), but not the images inside them.
- **Solution:** We fixed this by adding a wildcard (*/*.png*) to the end of the paths (eg.../Normal/*.png). This told the glob function to find all files ending in .png inside the folder.

G. Segmentation Issue (Only One Lung Detected)

- **Problem:** During the segmentation phase using Otsu's Thresholding and morphological operations, the algorithm successfully produced a binary mask. However, the final mask only included one region of the lung rather than both.
- **Mistake:** The morphological kernel used for cleaning was too large, which eroded part of one lung. Furthermore, the code only kept the two largest connected segments; since one lung became smaller after erosion, it was mistakenly removed. Global Otsu thresholding also struggled due to uneven brightness between the left and right lungs.
- **Solution:** We reduced the morphological kernel size to preserve thin lung edges while still removing small noise. We also added Gaussian smoothing before thresholding to even out illumination, and used a relative area threshold instead of fixed major two components. This allows both of the lungs to be correctly segmented/included and appear clearly in the final mask.

H. Segmentation Function Not Working on Dataset

- **Problem:** The segment_lungs_v2 and extract_lung_features functions worked perfectly on our testing image, but on the image dataset it skipped

the majority of the image, giving the message “Did not find exactly two lungs.”

- Mistake: The settings (like threshold values and size filters) were “overfitted” to the single test image. They were not general enough to work on the 4000+ images in the full dataset, which could have different contrasts, noises and sizes.
- Solution: This was solved by modifying the `extract_lung_features` function. We changed the `min_size` filter from a hard-coded value (like 5000) to be a percentage of the largest object (like $\text{np.max(sizes)} * 0.2$). This made the code adaptive, allowing it to correctly find lungs of different sizes and different images, which took our processed sample count from nearly 0 to 2700.

I. Black Outline of the Image Disrupting Bounding Boxes

- Problem: When doing the bounding boxes, it takes two halves of the entire image rather than the two lungs.
- Mistake: During the feature extraction of the image, there was a thick black border of the image that wasn't removed which caused the bounding boxes to be inaccurate (where it just takes the whole image rather than the two lungs).
- Solution: The thick black border was then cutout, allowing the lung regions to be specifically detected from the feature extraction (comparison shown in Figure 6 and 7).

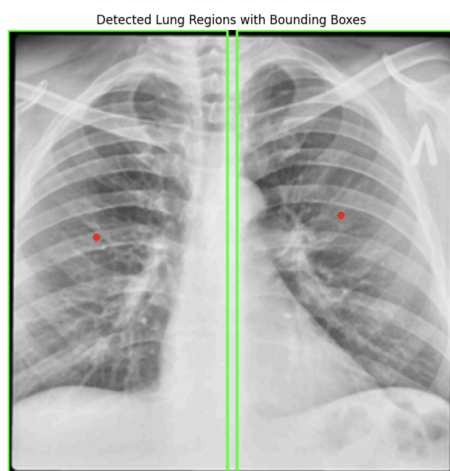


Figure 6: Initial Bounding Boxes (before black border removal)

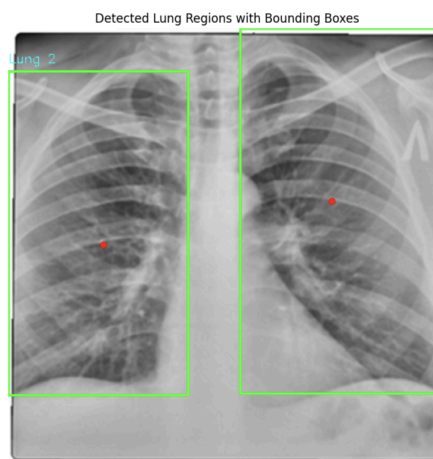


Figure 7: Improved Bounding Boxes

VI. RESULTS AND MODEL EVALUATION

This section details the last stage of the project which is the training of the machine learning model and the quantitative evaluation of its performance on an unseen test dataset.

J. Dataset Assembly and Preparation

The features extracted from both lungs were consolidated to create a final, structured dataset. For each X-ray image where two lungs were successfully segmented, a single feature vector was made. This vector comprised the five features which are area, perimeter, mean intensity, standard deviation and entropy. It comprised the five features for the left lung and the same five features for the right lung, and three additional “difference” features (`area_diff`, `entropy_diff`, `intensity_diff`). These different features were designed to capture asymmetries between the lungs, giving a strong indicator of unilateral lung disease.

Out of the 4,200 images in the original dataset, the image processing pipeline successfully processed and extracted features from 3,383 images. This final dataset was then split into a training set of 2,706 samples and a testing set of 677 samples. The model was trained exclusively on the training set, while the testing set was held out for final performance evaluation.

K. Model Training

A Random Forest Classifier, an ensemble learning method used due to its high performance and robustness on tabular data, was selected for the classification task. The model was configured with 100 individual decision trees and trained on the feature vectors from the 2,706 samples in the training set.


```
print(f"Training samples: {len(X_train)}")
print(f"Testing samples: {len(X_test)}")

• Total samples: 3383
  Training samples: 2706
  Testing samples: 677
```

Figure 8: Details of the samples from the dataset

```
Train the Random Forest Model

model = RandomForestClassifier(n_estimators=100)
print("Training the Random Forest model...")
model.fit(X_train, y_train)
print("Model training complete!")

Training the Random Forest model...
Model training complete!
```

Figure 9: Training the Model

L. Performance Evaluation

The trained model's ability to generalize was evaluated using the unseen test set of 677 images. The model achieved a strong **overall accuracy of 92.76%**. A more detailed breakdown of the performance will be given in the following figures.

```
--- Model Evaluation ---
Overall Accuracy: 92.76%

--- Classification Report ---
              precision    recall  f1-score   support

   Normal (0)             0.94      0.98      0.96         577
  Tuberculosis (1)        0.83      0.64      0.72         100

   accuracy                   0.93         677
  macro avg              0.89      0.81      0.84         677
 weighted avg              0.92      0.93      0.92         677

--- Confusion Matrix ---
[[564  13]
 [ 36  64]]
```

Figure 10: Model Evaluation & Confusion Matrix

The confusion matrix represents where the model predicted it right or wrong from the images. The top left (564) is the true negatives where 564 were actually "Normal" and the model predicted it correctly. Bottom right (64) is the true positives meaning 64 images were actually "Tuberculosis" and the model predicted it correctly. Then the top right (13) shows

the false positives where it's actually "Normal" but the model predicted wrongly. Bottom left (36) is false negative where the 36 images are actually "Tuberculosis" but the model predicted wrongly. This leads to 628 correct predictions out of 677 giving 92.76% overall accuracy.

VII. FRONT END IMPLEMENTATION

To improve usability and move beyond a notebook-based prototype, a front-end interface was implemented for the TB detection model. The interface provides an end-to-end workflow (upload >> analyze >> result) that enables non-technical users to run inference without interacting with code, reducing error and increasing accessibility. The results are displayed in a clear and responsible format, including the predicted class, confidence score, and extracted feature summaries to support interpretability and transparency. Overall, the front end demonstrates deployment readiness and helps validate the model in a realistic user interaction setting as a decision support tool rather than a definitive diagnostic system.

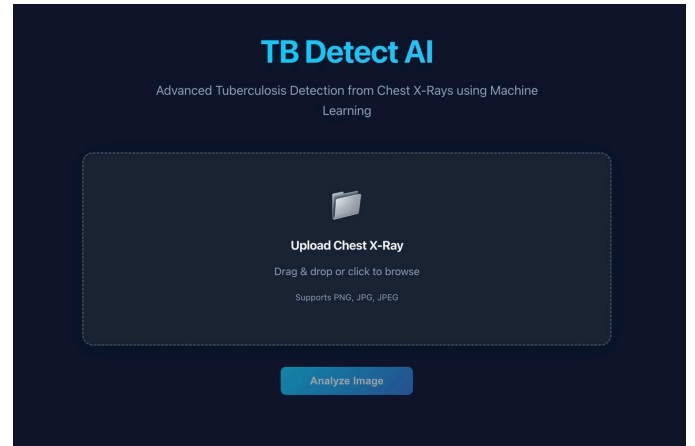


Figure 11: Upload the image

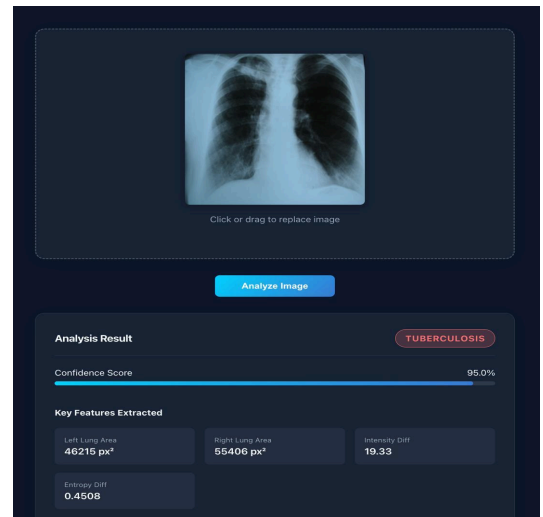


Figure 12: Results and Detection displayed

VIII. CONCLUSION

This paper successfully presents the design, implementation, and evaluation of a system for the automatic detection of tuberculosis in chest X-ray images. The methodology demonstrates that using a feature-based strategy, combining solid image processing with a standard machine learning classifier, can deliver high-performance results without requiring the heavy computational resources of deep learning models. Through careful pre-processing steps, segmenting the lung regions, and extracting meaningful geometric and texture features, the system can turn complex visual patterns into structured numerical data that the model can interpret effectively.

The trained Random Forest model achieved an overall accuracy of 92.76% on the unseen test set. These results show that the model performs very well at identifying healthy individuals, demonstrated by its 98 percent recall for the "Normal" class. This highlights its potential as a reliable screening tool to help rule out negative cases. The main weakness lies in its lower recall of 64 percent of the "Tuberculosis" class, which resulted in 36 false negatives out of 100 positive cases. Reducing false negatives is especially important in real clinical use, since missing a patient with tuberculosis can lead to delayed treatment and further spread of the disease. Although the model shows promising accuracy, minimizing such false negatives is the most critical key for clinical applications.

The segmentation phase proved to be the most sensitive component, as minor variations in thresholding and morphological filtering substantially influenced the number of processable samples. This suggests that the overall reliability of feature based pipelines is strongly constrained by segmentation robustness. Moreover, the hand crafted features used in this study particularly entropy, mean intensity, and asymmetry measures offered interpretability that would not be directly available from purely deep learning based systems, which supports potential clinical acceptance.

Future work may address the limitation in tuberculosis recall through several directions: (i) applying class-weighted or cost-sensitive training to penalize false negatives, (ii) incorporating additional discriminative features such as

Haralick texture descriptors or multi-scale gradient patterns, and (iii) leveraging hybrid designs that retain explicit feature extraction but replace the classifier with a lightweight neural model to improve sensitivity. Another promising direction is to integrate more adaptive segmentation methods to reduce preprocessing-stage failure.

In conclusion, the results demonstrate that a classical image processing pipeline combined with a feature-based machine learning model can provide a computationally efficient and interpretable solution for automated tuberculosis screening using chest X-ray images. With further refinement to reduce false negatives and improve segmentation stability, such systems have strong potential to support large-scale, low-cost prescreening in clinical practice, especially in resource-limited environments.

REFERENCES

- [1]"Tuberculosis (TB) Chest X-ray Database," *www.kaggle.com*. <https://www.kaggle.com/datasets/tawsifurrahman/tuberculosis-tb-chest-xray-dataset>
- [2]A. Shafi, "Sklearn Random Forest Classifiers in Python Tutorial," *www.datacamp.com*, Feb. 2023. <https://www.datacamp.com/tutorial/random-forests-classifier-python>
- [3]L. Breiman, "Random Forests," Jan. 2001. Available: <https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf>
- [4]Q. Wu *et al.*, "A color extraction algorithm by segmentation," *Scientific Reports*, vol. 13, no. 1, Dec. 2023, doi: <https://doi.org/10.1038/s41598-023-48689-y>.